

Robotics Integration Group Project 1

Lab 2 Experiment Report: ROS, TF and Homogeneous Transformations

Huang Boyu 24020036026. Lab group: Group 8.

Ocean University of China, 24th Cohort Sino-Foreign CS Class 1.

Lab date: 28th September 2026. Report date: 28th September 2026.

Summary

This report covers the completion of Lab 2 of the Robotics Integration Group Project. ROS 1 Noetic was installed on Ubuntu 20.04 (aarch64) and verified with turtlesim, including node–topic communication, the TF tree and RViz visualization. A `two_drones_pkg` was built that launches two drones: in the static scene two `static_transform_publisher` nodes publish the `world-to-av1/av2` transforms, while in the dynamic scene a `frames_publisher_node` publishes the time-varying transforms at 50 Hz, and a `plots_publisher_node` queries TF to display the world trajectories and the relative trajectory. Analytically, AV1 moves on a circle and AV2 on a parabola segment in the world frame, and the trajectory of AV2 in the AV1 frame is proven, using homogeneous transformations, to be an ellipse with semi-major axis $\sqrt{5}/2$ and semi-minor axis $1/2$ lying in the plane $z - 2y - 1 = 0$. The quaternion left- and right-multiplication operators are shown to be orthogonal and to commute, and intrinsic and extrinsic rotation sequences are shown to produce the same final attitude. The runtime observations agree with the mathematical derivations.

1 Objectives and Environment

This lab completes the installation of ROS Noetic, practices nodes, topics, launch files and TF tools, implements the publication of coordinate-frame transformations for two drones together with relative-trajectory queries, and proves with homogeneous transformations that the relative trajectory is a planar ellipse.

Table 1: Actual experiment environment.

Item	Actual environment
Hardware	MacBook Air M5, VMware virtual machine
Operating system	Ubuntu 20.04.5 LTS, aarch64 (ARM64), hostname <code>ubuntu2004</code>
ROS	ROS 1 Noetic Desktop-Full, <code>ros_comm</code> 1.17.4
Build & dependency tools	<code>catkin_tools</code> 0.9.4, <code>roscpp</code> 0.25.1
Python	3.8.10
Workspace	<code>/home/hby/vnav_ws</code>
Material organization	Screenshots and the report were organized on another host <code>hby-VMware20-1</code> ; all ROS results come from <code>ubuntu2004</code>
Course code baseline	MIT-SPARK/VNAV-labs, commit <code>609f31fec484daafef6207ce283d23b424a2072f</code> (ROS 1)
Lab source commit	Personal repository commit <code>03918a9</code>

The lab follows the 2023 ROS 1 handouts. Since the current default branch of the course repository uses ROS 2 `ament_cmake`, the ROS 1 historical version above is adopted. This directory contains the complete package, 7 raw experiment screenshots and this report. Formulas are typeset in LaTeX and can be read on GitHub.

2 ROS Installation and Introductory Verification

2.1 Installation

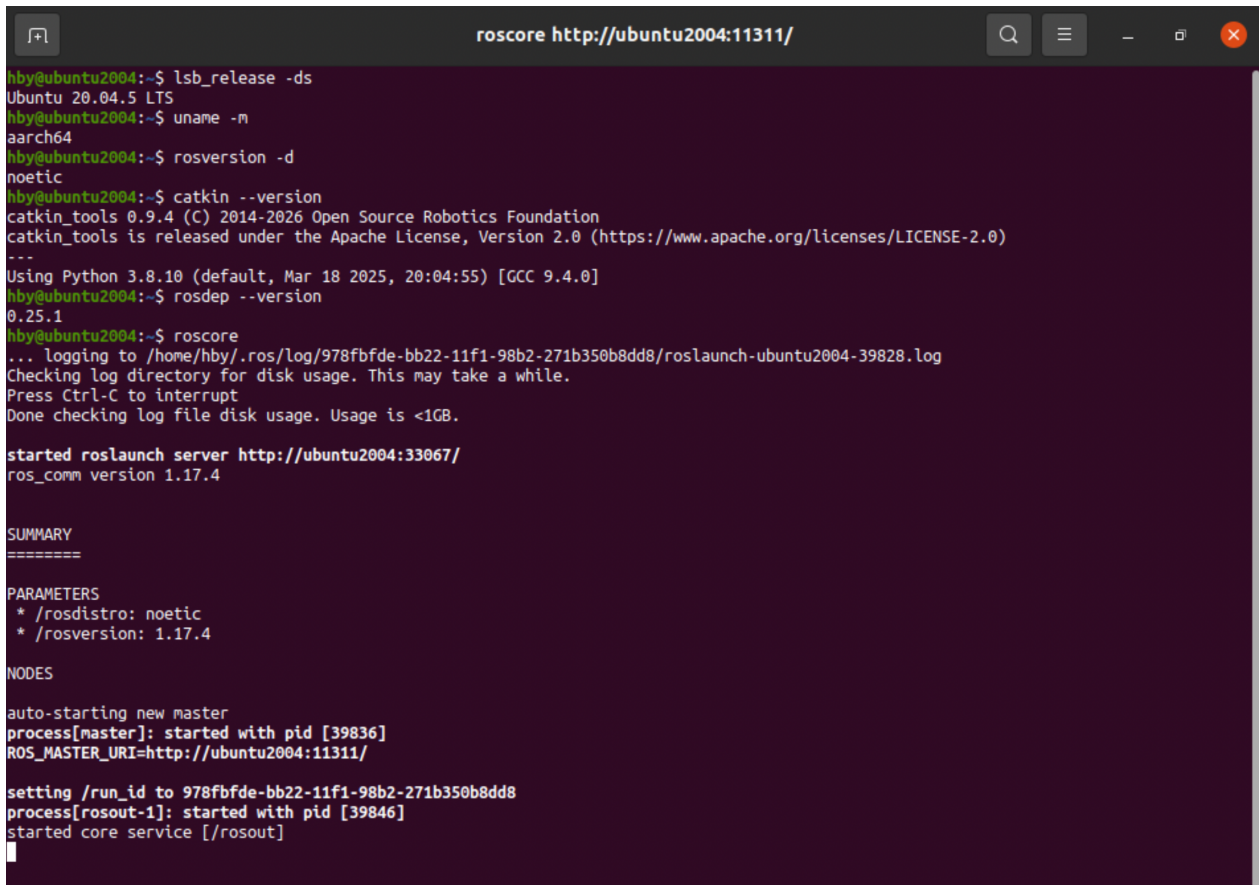
After confirming that the system is Ubuntu focal on aarch64, the ROS ARM64 apt repository and signing key were configured and `ros-noetic-desktop-full` was installed. The following line was added to `~/.bashrc`:

```
source /opt/ros/noetic/setup.bash
```

Build dependencies were installed and `rosdep` was initialized:

```
sudo apt install python3-rosdep python3-rosinstall python3-rosinstall-generator python3-vcsto
sudo rosdep init
rosdep update --include-eol-distros
```

The update output contains Add distro "noetic" and the cache was written successfully. `rosversion -d` returns `noetic`, and `roscore` prints `started core service [/rosout]`, showing that the core service starts normally.



```
roscore http://ubuntu2004:11311/

hby@ubuntu2004:~$ lsb_release -ds
Ubuntu 20.04.5 LTS
hby@ubuntu2004:~$ uname -m
aarch64
hby@ubuntu2004:~$ rosversion -d
noetic
hby@ubuntu2004:~$ catkin --version
catkin_tools 0.9.4 (C) 2014-2026 Open Source Robotics Foundation
catkin_tools is released under the Apache License, Version 2.0 (https://www.apache.org/licenses/LICENSE-2.0)
...
Using Python 3.8.10 (default, Mar 18 2025, 20:04:55) [GCC 9.4.0]
hby@ubuntu2004:~$ rosdep --version
0.25.1
hby@ubuntu2004:~$ roscore
... logging to /home/hby/.ros/log/978fbfde-bb22-11f1-98b2-271b350b8dd8/roslaunch-ubuntu2004-39828.log
Checking log directory for disk usage. This may take a while.
Press Ctrl-C to interrupt
Done checking log file disk usage. Usage is <1GB.

started roslaunch server http://ubuntu2004:33067/
ros_comm version 1.17.4

SUMMARY
=====
PARAMETERS
* /rostdistro: noetic
* /rosversion: 1.17.4

NODES

auto-starting new master
process[master]: started with pid [39836]
ROS_MASTER_URI=http://ubuntu2004:11311/

setting /run_id to 978fbfde-bb22-11f1-98b2-271b350b8dd8
process[rosout-1]: started with pid [39846]
started core service [/rosout]
```

Figure 1: ROS installation and running verification

Figure 01: Ubuntu version, CPU architecture, ROS and development tool versions, and the ROS Master startup result.

2.2 Nodes and Topics

`roscore`, `roslaunch turtlesim turtlesim_node` and `roslaunch turtlesim turtle_teleop_key` were run separately, and the turtle was controlled with the arrow keys. `rostopic list` shows `/rosout`, `/turtlesim` and `/teleop_turtle`.

`roslaunch rqt_graph rqt_graph` was used to inspect the communication relationships, and `rostopic echo /turtle1/cmd_vel` to inspect the messages:

```
/teleop_turtle → /turtle1/cmd_vel → /turtlesim
```

The keyboard node publishes velocity commands and the turtle node subscribes to and executes them. In the message, `linear.x` controls forward/backward motion and `angular.z` controls turning. The ROS Master provides naming and registration services that help nodes discover each other; the actual data is transferred directly between nodes.

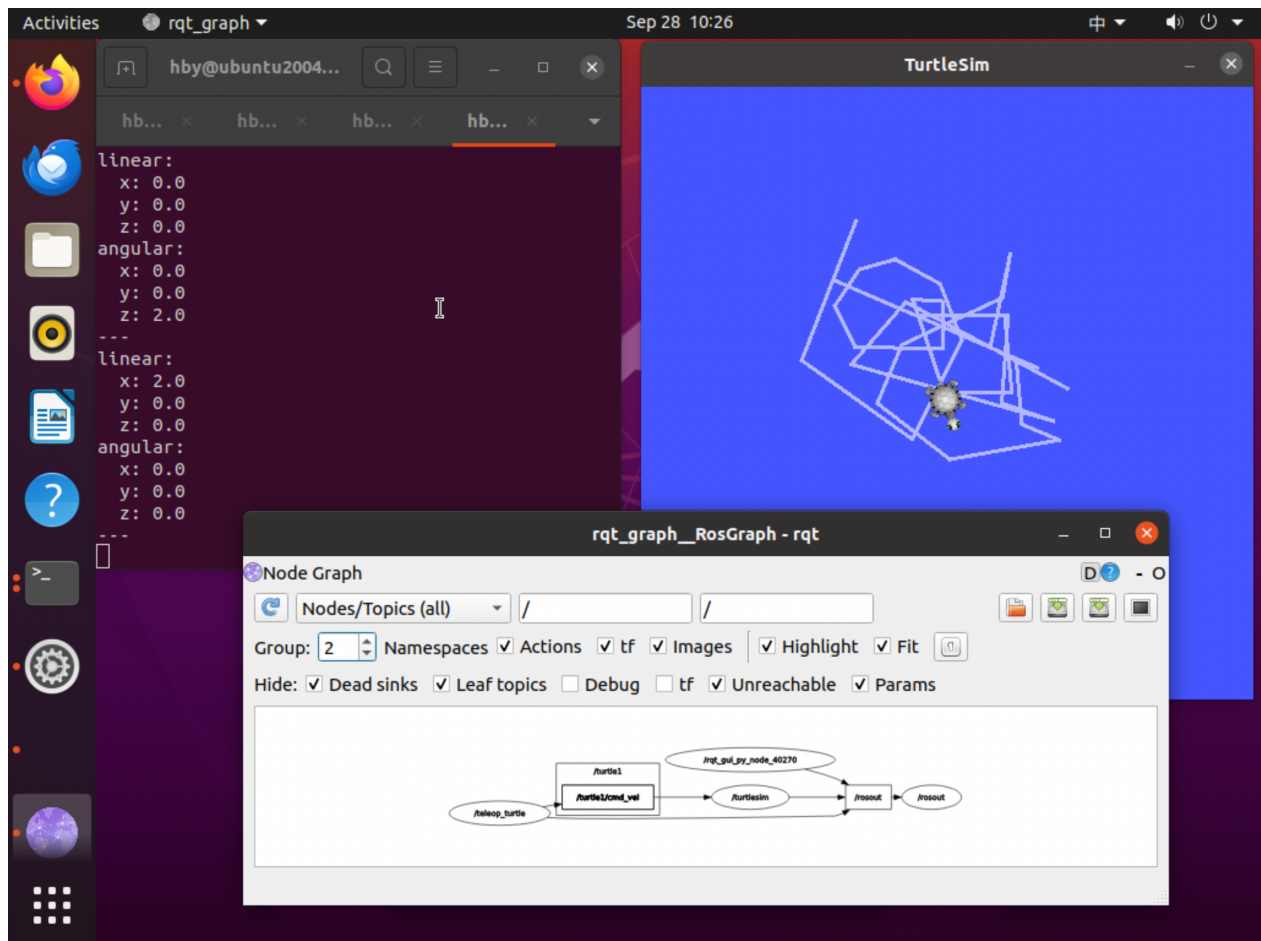


Figure 2: Turtle control and node/topic communication

Figure 02: Turtle motion trajectory, velocity messages, and the node-topic communication graph.

2.3 TF and RViz

The following tools were run (each long-running program in its own terminal):

```
roslaunch turtle_tf turtle_tf_demo.launch
roslaunch rqt_tf_tree rqt_tf_tree
roslaunch tf_echo turtle1 turtle2
rviz
```

It was observed that the second turtle follows the controlled turtle. The TF tree connects `world` to `turtle1` and `turtle2` respectively. `tf_echo turtle1 turtle2` gives the position and orientation of `turtle2` in the `turtle1` frame. At the moment of the screenshot the two positions almost coincide, yet the relative yaw is about -63.21° , which shows that coincident positions do not imply identical attitudes.

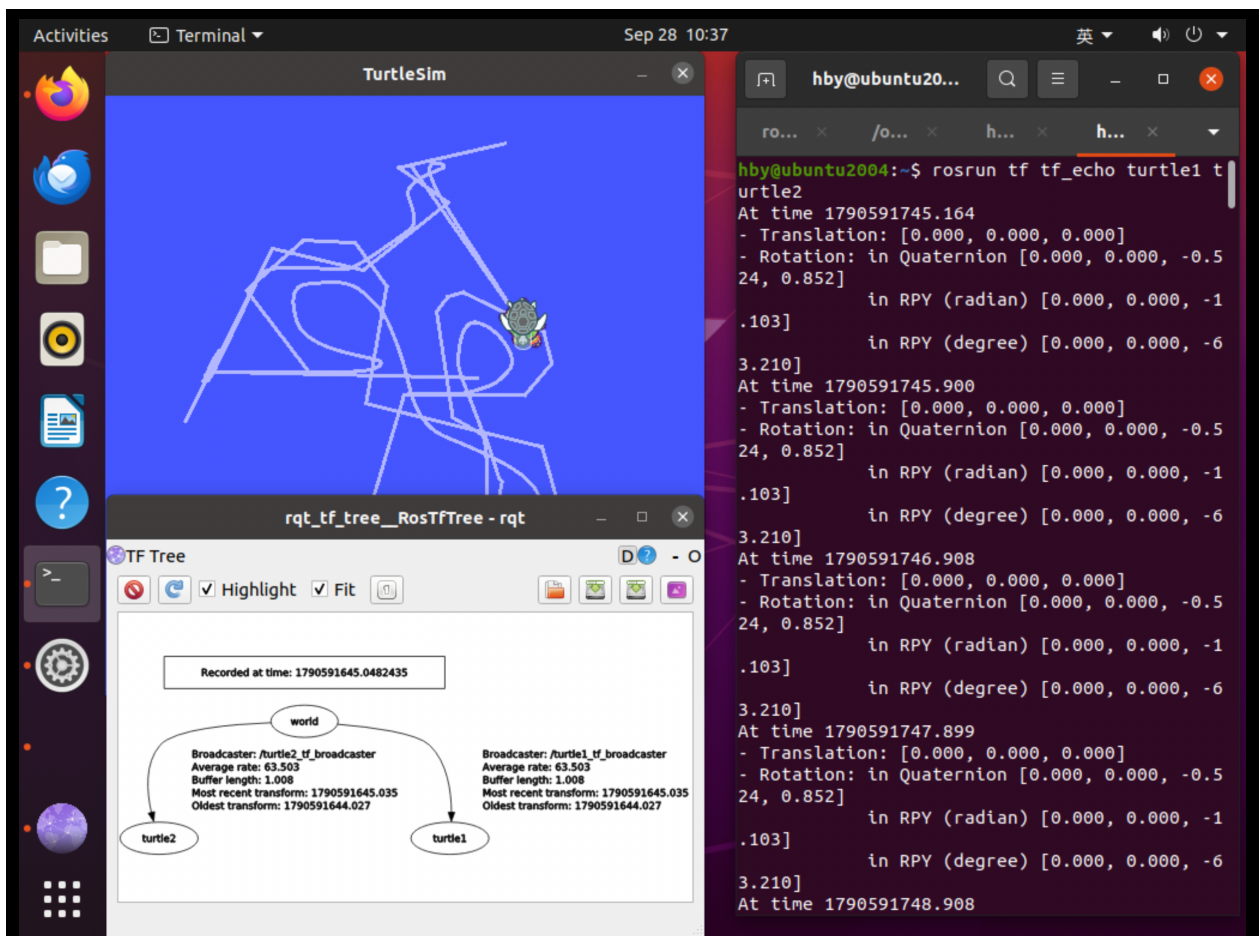


Figure 3: Two-turtle following and TF relationships

Figure 03: Two-turtle following, TF tree, and the relative transform output.

In RViz, Fixed Frame was set to **world**, TF was added and Show Names was enabled. The red, green and blue axes correspond to x, y and z respectively. Zooming out reveals all coordinate frames; the TF status is OK.

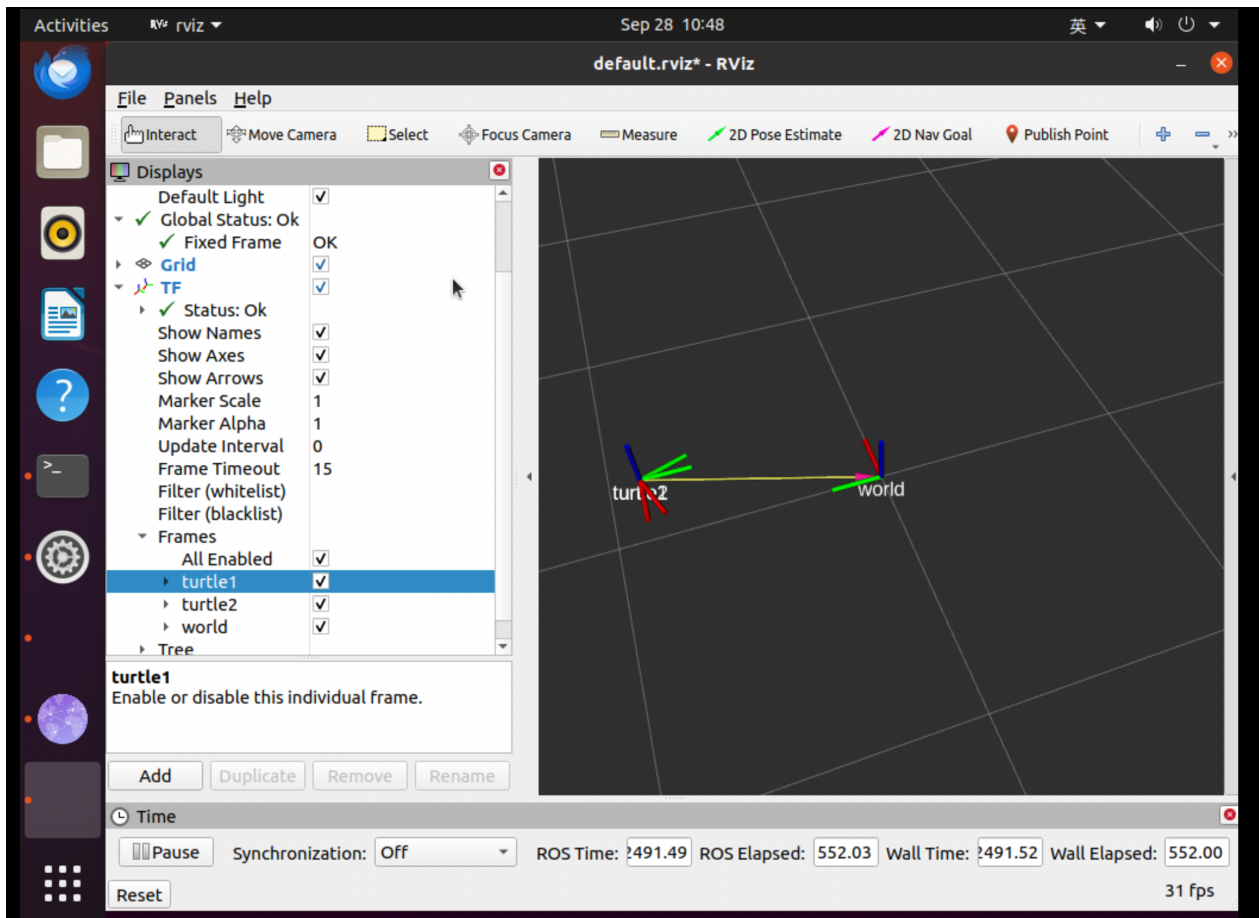


Figure 4: RViz frame visualization

Figure 04: Visualization of the **world** frame and the two turtle frames; when the turtle follows to a nearby position, its name and axes may overlap.

3 Workspace and Reproduction

`~/vnav_ws` was initialized, the course ROS 1 packages were copied into `src`, and `catkin build` was run. The actual output contains `Finished <<< two_drones_pkg` and `All 1 packages succeeded!` with no packages skipped.

In an environment with ROS Noetic and the course dependencies installed, the results can be reproduced from this repository. The following assumes the personal repository is at `~/vnav-personal` and the target workspace does not yet contain a package of the same name:

```
source /opt/ros/noetic/setup.bash
mkdir -p ~/vnav_ws/src
cd ~/vnav_ws
catkin init
cp -a ~/vnav-personal/lab2/two_drones_pkg ~/vnav_ws/src/
catkin build
source ~/vnav_ws/devel/setup.bash
```

```
roslaunch two_drones_pkg two_drones.launch static:=True
```

After stopping the static scene, launch the dynamic scene without `static:=True`:

```
roslaunch two_drones_pkg two_drones.launch
```

A new terminal needs `source ~/vnav_ws/devel/setup.bash` again. `roslaunch` starts the Master automatically when none is running. In RViz enable `MarkerArray` and `TF`; set Fixed Frame to `world` and `av1` in turn to observe the results.

4 Deliverable 1: Nodes, Topics and Launch Files

4.1 Nodes of the Static Scene

Actual `rostopic list` output:

```
/av1broadcaster  
/av2broadcaster  
/plots_publisher_node  
/rosout  
/rviz
```

Apart from the logging node `/rosout`, which the problem statement allows to be ignored, there are four functional nodes. The two static publisher nodes publish the transforms from `world` to `av1` and `av2` respectively; the positions are `(1,0,0)` and `(0,0,1)`, both with quaternion `(0,0,0,1)`.

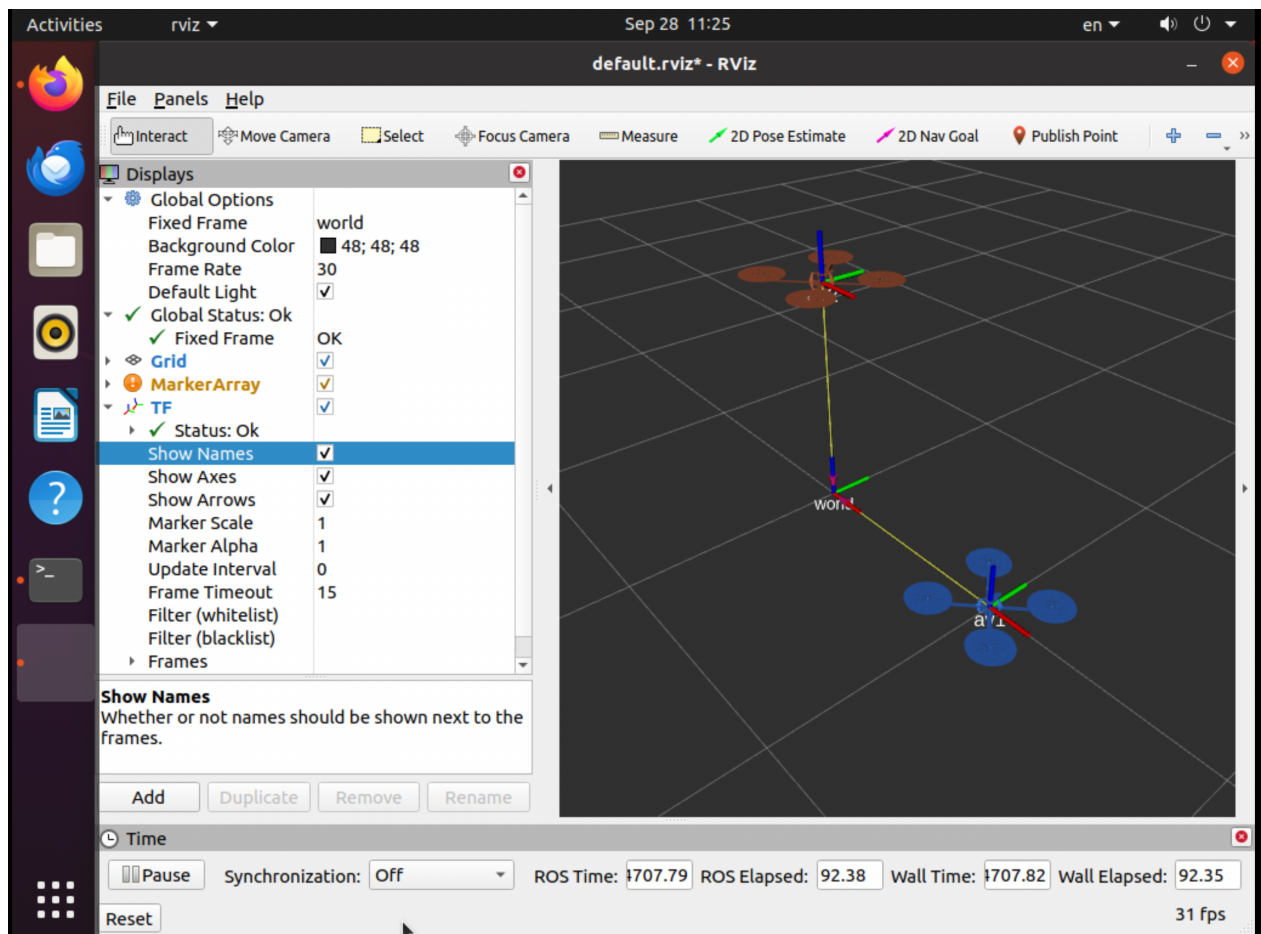


Figure 5: Static scene with two drones

Figure 05: Blue AV1, red (rendered orange-red) AV2, and the TF axes. This image was captured before the trajectory-marker warning was fixed; TF and the global status are OK, and the orange MarkerArray warning was resolved in a later code modification.

4.2 Equivalent Launch Without roslaunch

First stop the scene started by the launch file, then run the following commands in five independent terminals, each with the ROS environment sourced beforehand. It was verified experimentally that these commands reproduce the two stationary drones.

```
# Terminal 1
roscore

# Terminal 2
roslaunch tf2_ros static_transform_publisher 1 0 0 0 0 0 1 world av1 __name:=av1broadcaster

# Terminal 3
roslaunch tf2_ros static_transform_publisher 0 0 1 0 0 0 1 world av2 __name:=av2broadcaster

# Terminal 4
source ~/vnav_ws/devel/setup.bash
roslaunch two_drones_pkg plots_publisher_node

# Terminal 5
source ~/vnav_ws/devel/setup.bash
roslaunch rviz rviz -d "$(rospack find two_drones_pkg)/config/default.rviz" __name:=rviz
```

During manual verification RViz was started without `__name`, which produced an RViz node with an auto-generated name; the name argument above is added so that the node name also matches the launch file.

4.3 Publishing and Subscription Relationships

`roslaunch info` and `rostopic info /visuals` give the following results; the `/rosout` log topics published by all nodes are omitted from the table:

Table 2: Published and subscribed topics of the static-scene nodes.

Node	Published topics	Subscribed topics
/av1broadcaster	/tf_static	none
/av2broadcaster	/tf_static	none
/plots_publisher_node	/visuals	/tf, /tf_static
/rviz	/clicked_point, /initialpose, /move_base_simple/goal	/tf, /tf_static, /visuals

The message type of `/tf_static` is `tf2_msgs/TFMessage`; `/visuals` is `visualization_msgs/MarkerArray`, published by `/plots_publisher_node` and subscribed to by `/rviz`. Hence the drone mesh models come from `/visuals`, and their spatial positions are determined by the TF relationships. RViz's interactive topics only emit messages when the corresponding tools are used; they do not transmit continuously.

4.4 What Changes When `static:=True` Is Omitted

The default value of the `static` argument in the launch file is `false`. `<group if="$(arg static)">` starts the two static publishers only when the argument is true; `unless="$(arg static)"` starts `/frames_publisher_node` when the argument is false.

Therefore, when the argument is omitted, the two static publisher nodes are replaced by the dynamic publisher node, while the plots node and RViz still start. With the template motion code unfilled there is no correct dynamic transform; after completing Deliverable 2, the two drones move along the specified trajectories. After closing the leftover RViz, the functional nodes of the dynamic scene are `/frames_publisher_node`, `/plots_publisher_node`, `/rviz`, plus the logging node `/rosout`.

5 Deliverable 2: Publishing the Motion Transforms

Source: `two_drones_pkg/src/frames_publisher_node.cpp`.

The node runs its callback on a 0.02 s timer (nominally 50 Hz) and uses the seconds elapsed since node startup as the time variable; both transforms share the same timestamp, the parent frame is `world` in both cases, and the child frames are `av1` and `av2`.

$$o_1^w(t) = (\cos t, \sin t, 0)^T, \quad o_2^w(t) = (\sin t, 0, \cos 2t)^T. \quad (1)$$

AV1 uses `q.setRPY(0.0, 0.0, time)`; AV2 uses the identity quaternion. The rotation matrix of AV1 is:

$$R_1^w(t) = \begin{bmatrix} \cos t & -\sin t & 0 \\ \sin t & \cos t & 0 \\ 0 & 0 & 1 \end{bmatrix}. \quad (2)$$

Its second column, the world-frame direction of AV1's y-axis, is $(-\sin t, \cos t, 0)^T$, which equals the time derivative of its position; the third column is the world z-axis. Hence the requirement is satisfied that the y-axis points along the direction of motion and the z-axis stays parallel to the world z-axis. Finally `broadcaster.sendTransform` publishes both transforms.

After compiling and launching, the two drones start moving. This verifies the kinematic motion only, without considering the dynamical feasibility of a real quadrotor; 50 Hz is the value set in code, and no separate measurement of the actual frequency was performed.

6 Deliverable 3: Querying Transforms and Displaying Trajectories

Source: `two_drones_pkg/src/plots_publisher_node.cpp`. The core statement that was filled in:

```
transform = parent->tf_buffer.lookupTransform(  
    ref_frame, dest_frame, ros::Time(0));
```

The first argument is the reference frame in which the result is expressed (target), the second is the frame of the target object (source); the returned transform converts coordinates in `dest_frame` into `ref_frame`. Its translation part is the position of the target frame's origin in the reference frame. `ros::Time(0)` queries the latest available transform; it does not query the simulation start time.

The queries `(world,av1)`, `(world,av2)` and `(av1,av2)` yield the blue world trajectory, the red world trajectory and the red dashed relative trajectory. Historical points are stored in their own reference frames: the AV1-relative trajectory marker uses `av1` as its `frame_id`, so in the world view it transforms together with the current AV1 attitude and must not be mistaken for AV2's world history.

Two fixes were also made: setting `pose.orientation.w = 1.0` on the trajectory markers eliminates the zero-quaternion warning; declaring `tf_buffer` before `tf_listener` guarantees that the object referenced by the listener is constructed first.

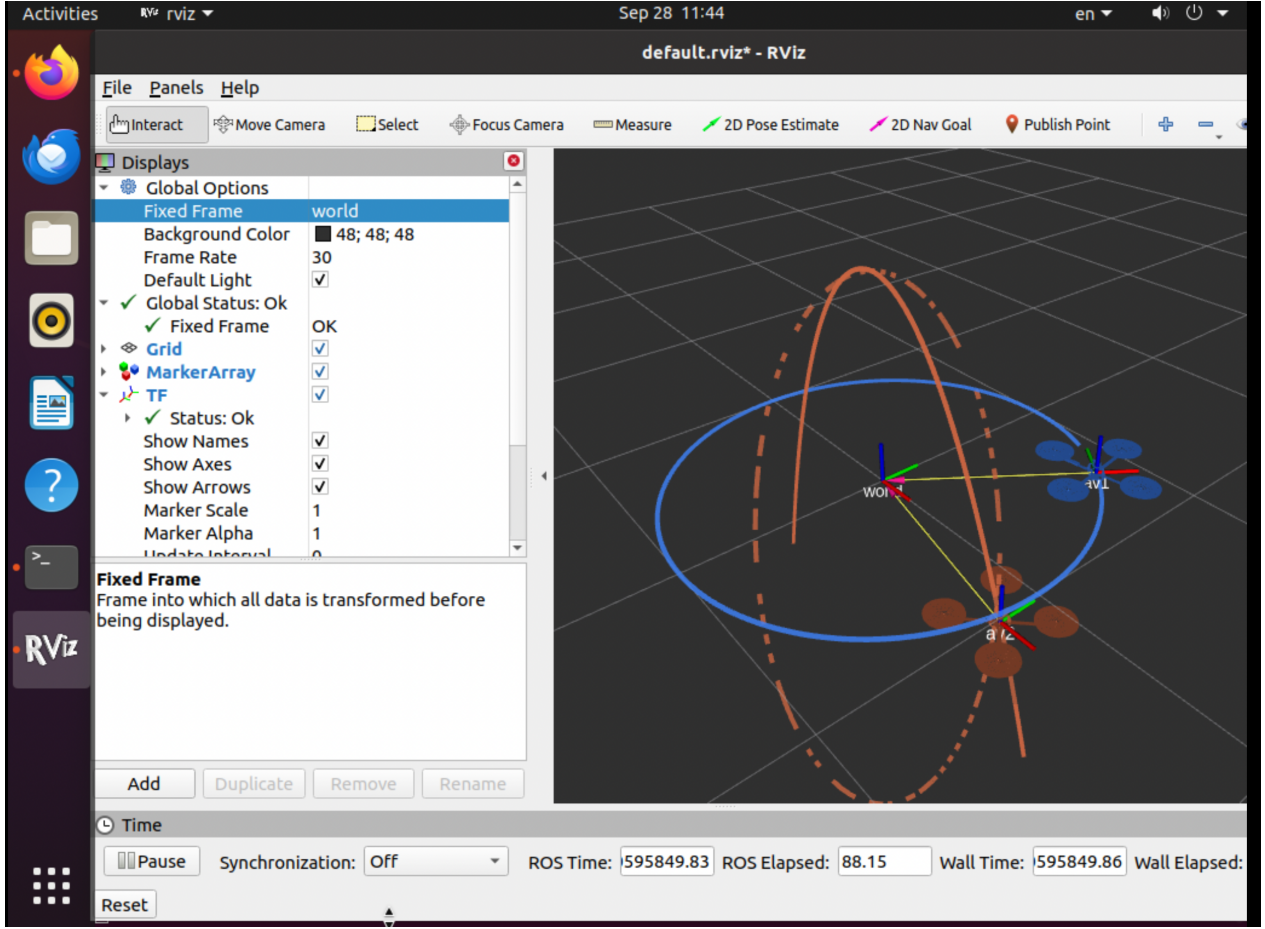


Figure 6: Drone trajectories in the world frame

Figure 06: Fixed Frame = world. The blue solid line is AV1's circular trajectory, the orange-red solid line is AV2's parabola segment, and the dashed line is the relative trajectory converted into the current world view. The screenshot edge crops part of the dashed line, but the two world trajectories and the status are recognizable.

Figure 07: Fixed Frame = av1. AV1 is stationary in this reference frame and AV2's dashed trajectory appears as a tilted planar ellipse. The global and TF statuses are normal, and the original MarkerArray quaternion warning has been eliminated.

The trajectories use finite buffers: 300 points for each world trajectory and 160 for the relative trajectory, covering about 6 s and 3.2 s respectively at the nominal 50 Hz. The period of the world circle is 2π seconds, so the blue circle in the screenshot may show a small gap; the relative ellipse has period π seconds. Finite sampling and view projection do not change the geometric nature of the analytic trajectories.

7 Deliverable 4: Homogeneous Transformations and the Ellipse Derivation

Convention: ${}^A T_B$ converts coordinates in frame B into frame A; column vectors are used; write $c = \cos t$, $s = \sin t$.

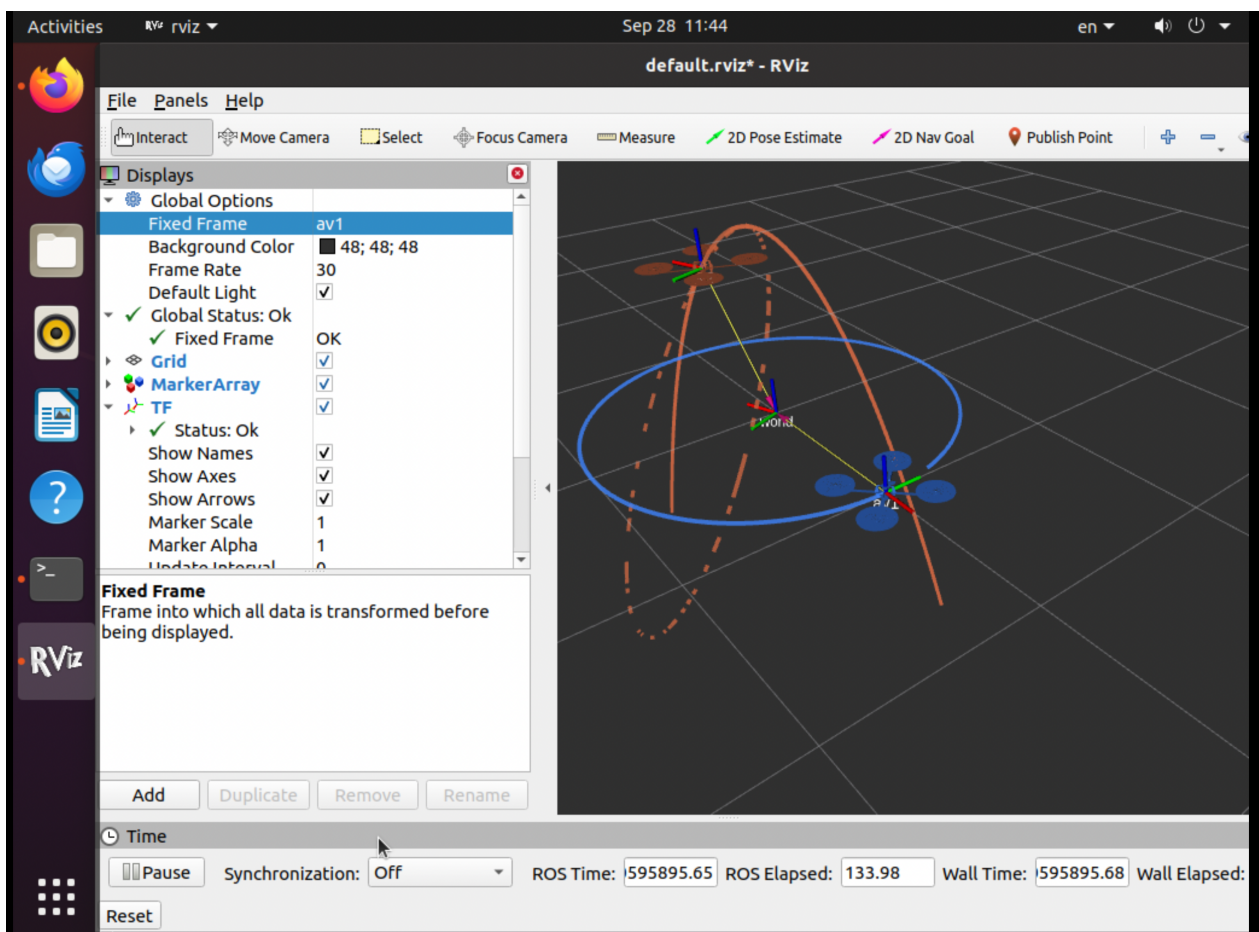


Figure 7: Relative elliptical trajectory in the AV1 frame

7.1 AV2's World Trajectory Is a Parabola Segment

From $x_w = \sin t$, $y_w = 0$, $z_w = \cos 2t = 1 - 2\sin^2 t$, eliminating time gives (3):

$$\boxed{y_w = 0, \quad z_w = 1 - 2x_w^2, \quad -1 \leq x_w \leq 1.} \quad (3)$$

This is a downward-opening parabola segment in the x-z plane; AV2 moves back and forth along it over time.

7.2 Position of AV2 Relative to AV1

The two world homogeneous transforms are (4):

$${}^wT_1 = \begin{bmatrix} c & -s & 0 & c \\ s & c & 0 & s \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad {}^wT_2 = \begin{bmatrix} 1 & 0 & 0 & s \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & \cos 2t \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (4)$$

Using the rigid-body inverse formula (5):

$$\begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix}^{-1} = \begin{bmatrix} R^T & -R^T p \\ 0 & 1 \end{bmatrix}, \quad {}^1T_W = \begin{bmatrix} c & s & 0 & -1 \\ -s & c & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (5)$$

Composing:

$${}^1T_2 = {}^1T_W {}^wT_2 = \begin{bmatrix} c & s & 0 & cs - 1 \\ -s & c & 0 & -s^2 \\ 0 & 0 & 1 & \cos 2t \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (6)$$

Hence the relative position is (7):

$$\boxed{o_2^1(t) = \begin{bmatrix} \frac{1}{2} \sin 2t - 1 \\ \frac{1}{2} \cos 2t - \frac{1}{2} \\ \cos 2t \end{bmatrix}.} \quad (7)$$

7.3 The Plane of the Trajectory

Denote the position in the AV1 frame by (x, y, z) . From $y = (\cos 2t - 1)/2$ and $z = \cos 2t$:

$$\boxed{\Pi : z - 2y - 1 = 0.} \quad (8)$$

Thus the whole trajectory lies in a fixed plane; a normal vector can be taken as $(0, -2, 1)^T$.

7.4 Constructing a Coordinate System on the Plane

Take the center $p^1 = (-1, -1/2, 0)^T$ and choose the right-handed orthonormal basis (9):

$$e_u = (1, 0, 0)^T, \quad e_v = (0, 1, 2)^T/\sqrt{5}, \quad e_n = (0, -2, 1)^T/\sqrt{5}. \quad (9)$$

$e_u \times e_v = e_n$, and both e_u and e_v lie in the trajectory plane. The transform from the new frame P to AV1 and its inverse are (10):

$${}^1T_P = \begin{bmatrix} 1 & 0 & 0 & -1 \\ 0 & 1/\sqrt{5} & -2/\sqrt{5} & -1/2 \\ 0 & 2/\sqrt{5} & 1/\sqrt{5} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad {}^PT_1 = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1/\sqrt{5} & 2/\sqrt{5} & 1/(2\sqrt{5}) \\ 0 & -2/\sqrt{5} & 1/\sqrt{5} & -1/\sqrt{5} \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (10)$$

Applied to the homogeneous coordinates of the relative position (11):

$$\begin{bmatrix} u \\ v \\ n \\ 1 \end{bmatrix} = {}^PT_1 \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} \frac{1}{2} \sin 2t \\ \frac{\sqrt{5}}{2} \cos 2t \\ 0 \\ 1 \end{bmatrix}. \quad (11)$$

The normal component is always zero; the planar 2-D coordinates are (u, v) and the trajectory center has been moved to the origin.

7.5 The Ellipse and Its Semi-Axes

Eliminating time with the sum-of-squares identity gives (12):

$$\boxed{\frac{u^2}{(1/2)^2} + \frac{v^2}{(\sqrt{5}/2)^2} = 1.} \quad (12)$$

The curve is symmetric about the u - and v -axes, i.e., an ellipse centered at the origin. The semi-major axis lies along e_v with length $\sqrt{5}/2 \approx 1.118$; the semi-minor axis lies along e_u with length $1/2 = 0.5$. This matches the relative trajectory in Figure 07.

8 Deliverable 5: Properties of Quaternions

The problem's convention is adopted: the vector part comes first and the scalar part last: $q = (v, w)$ with $v = (q_1, q_2, q_3)^T$ and $w = q_4$. Let $V = [v]_\times$, i.e., $Va = v \times a$. Then the two matrix operators are (13):

$$\Omega_1(q) = \begin{bmatrix} wI_3 + V & v \\ -v^T & w \end{bmatrix}, \quad \Omega_2(q) = \begin{bmatrix} wI_3 - V & v \\ -v^T & w \end{bmatrix}. \quad (13)$$

Here $\Omega_1(q)p = q \otimes p$ and $\Omega_2(q)p = p \otimes q$. The conjugate is $\bar{q} = (-v, w)$, hence $\Omega_i(q)^T = \Omega_i(\bar{q})$.

8.1 Orthogonality and Intuitive Interpretation

Let $M_\sigma = \begin{bmatrix} wI_3 + \sigma V & v \\ -v^T & w \end{bmatrix}$ with $\sigma = \pm 1$ for the two operators. Using $V^T = -V$, $Vv = 0$ and $V^2 = vv^T - \|v\|^2 I_3$:

$$M_\sigma^T M_\sigma = \begin{bmatrix} w^2 I_3 - V^2 + vv^T & -\sigma V v \\ \sigma v^T V & w^2 + v^T v \end{bmatrix} = (w^2 + \|v\|^2) I_4. \quad (14)$$

A unit quaternion satisfies $w^2 + \|v\|^2 = 1$, so $M_\sigma^T M_\sigma = I_4$. The matrix is square, so its inverse equals its transpose, and also $M_\sigma M_\sigma^T = I_4$; both operators are orthogonal.

Intuitively, the quaternion norm is multiplicative, so left- or right-multiplication by a unit quaternion preserves the length of any 4-D vector; such a linear isometry corresponds to an orthogonal matrix.

8.2 Conversion to the Unit Rotation Quaternion

For a unit q , $\bar{q} \otimes q = q \otimes \bar{q} = e$ with $e = (0, 0, 0, 1)^T$, hence (15):

$$\boxed{\Omega_1(q)^T q = \bar{q} \otimes q = e, \quad \Omega_2(q)^T q = q \otimes \bar{q} = e.} \quad (15)$$

The unit rotation corresponds to “a rotation and its inverse canceling each other”.

8.3 Commutation of the Two Operators

For arbitrary $x, y, p \in \mathbb{R}^4$, by associativity of quaternion multiplication (16):

$$\begin{aligned} \Omega_1(x)\Omega_2(y)p &= x \otimes (p \otimes y) \\ &= (x \otimes p) \otimes y = \Omega_2(y)\Omega_1(x)p. \end{aligned} \quad (16)$$

Since p is arbitrary (17):

$$\boxed{\Omega_1(x)\Omega_2(y) = \Omega_2(y)\Omega_1(x).} \quad (17)$$

Replacing y by its conjugate and using $\Omega_2(y)^T = \Omega_2(\bar{y})$ (18):

$$\boxed{\Omega_1(x)\Omega_2(y)^T = \Omega_2(y)^T \Omega_1(x).} \quad (18)$$

Here x and y need not be unit quaternions; what commutes is the left- and right-multiplication operators, not quaternion multiplication itself.

9 Deliverable 6 (Optional): Intrinsic and Extrinsic Rotations

Let the attitude matrix R convert body-frame coordinates into world coordinates. For a body-frame vector p , its world coordinates are Rp . Applying a rotation A about a fixed world axis gives $p' = A(Rp)$, so the new attitude is AR : left multiplication.

A rotation B about a body-local axis corresponds in the world frame to the operator RBR^{-1} , so the new attitude is $(RBR^{-1})R = RB$: right multiplication.

Starting from the same aligned attitude $R = I$, the extrinsic sequence applied in the order R_0, R_1, R_2 ends at $R_2 R_1 R_0$; the intrinsic sequence applied in the same order R_0, R_1, R_2 also ends

at $R_2R_1R_0$. Hence the two final attitudes are identical. For any n rotations, the same reasoning yields $R_{n-1} \cdots R_1R_0$ in both cases.

The problem's example is $R_0 = R_x(90^\circ)$, $R_1 = R_y(180^\circ)$, $R_2 = R_x(-30^\circ)$, and the common result is (19):

$$R_2R_1R_0 = \begin{bmatrix} -1 & 0 & 0 \\ 0 & -1/2 & -\sqrt{3}/2 \\ 0 & -\sqrt{3}/2 & 1/2 \end{bmatrix}. \quad (19)$$

This same-axis labeling and reversed-order equivalence presuppose that the initial local axes and the fixed axes are aligned. If the initial attitude is arbitrary, the two final products are $R_{n-1} \cdots R_0R_{\text{init}}$ and $R_{\text{init}}R_{n-1} \cdots R_0$ respectively, which cannot be asserted to be equal outright.

10 Problems Encountered and Conclusions

Table 3: Problems encountered during the lab and how they were handled.

Problem	Cause and handling
apt-source configuration command errored	Two commands were concatenated into <code>.ascecho</code> ; after splitting them, the repository and key configuration succeeded
catkin detected the home directory as a workspace	A <code>.catkin_tools</code> directory existed in home; it was renamed as a backup, then the workspace was initialized in <code>~/vnav_ws</code>
Build reported “succeeded” but skipped the lab package	The default source uses <code>ament_cmake</code> ; after switching to the ROS 1 historical version, <code>two_drones_pkg</code> was actually compiled
RViz initially showed only <code>world</code>	Zooming out and checking the TF Frames displays all coordinate frames
Static MarkerArray quaternion warning	Trajectory markers default to all-zero orientation; setting <code>orientation.w=1.0</code> removes it
Leftover blank RViz window	Close the old standalone RViz, keep the instance started by the current launch, and check the node list
Modified-script content mismatch	Inspection showed the three edits were already present, so redundant replacement was avoided; recompiled and reran directly

The lab completed the ROS installation, node-topic communication, and TF-tree and RViz verification; it implemented and ran two dynamic transforms and three trajectories. The runtime observations and the mathematical derivation agree: AV1 traces a circle in the world frame and AV2 traces a parabola segment; AV2's trajectory in the AV1 frame lies in the plane (8) and is an ellipse with semi-axes $\sqrt{5}/2$ and $1/2$. The screenshots are evidence of actual operation; the mathematical proofs are analytic derivations, and the screenshot observations were not treated as a quantitative error test.

References

1. [MIT VNAV 2023 — Installing ROS](#)

2. [MIT VNAV 2023 — Introduction to ROS](#)
3. [MIT VNAV 2023 — Lab 2 Exercises](#)
4. [MIT-SPARK/VNAV-labs ROS 1 source baseline](#)